HashiCorp Certified: Terraform Associate（004）　目標 4e　#08

式と関数

条件式、for 式とグループ化、スプラット式、文字列テンプレート、dynamic ブロック、組みこみ関数のしくみ（展開・console・外の状態を使う関数）、よく使う関数（lookup・merge・join・split・element・coalesce・try・can・flatten・zipmap・cidrsubnet）

各ページの下に、根拠にした資料（HashiCorp の Exam Content List（004）、Terraform ドキュメント v1.12.x の Conditional Expressions・for Expressions・Splat Expressions・Strings and Templates・dynamic Blocks・Function Calls・Built-in Functions の各関数のページ）を書いた。英語の原文は『』で引用した

第1章

式

値を選ぶ・変換する・くり返す。

条件式と for 式

```
var.a != "" ? var.a : "default-a"
# 2 つの値は同じ型にそろえる
var.example ? 12 : "hello"    # いつも文字列
var.example ? tostring(12) : "hello"
```

condition ? 真のとき : 偽のとき。型があいまいなら tostring などで明示する。

```
[for s in var.list : upper(s)]          # タプル
{for s in var.list : s => upper(s)}     # オブジェクト
[for s in var.list : s if s != ""]      # 絞りこみ
[for i, v in var.list : "${i} is ${v}"] # 番号と値
{for s in var.users : s.role => s.name...}
# ... でグループ化（キーごとのリスト）
```

かこみ	結果
[ ]	タプル（値の式 1 つ）
{ }	オブジェクト（キー => 値）。キーが重なるとエラー
{ … ... }	グループ化モード：同じキーの値をリストに

マップ・オブジェクトの入力では、2 つの一時変数はキーと値。リスト・タプルでは番号と値。

マップ・オブジェクトはキーの辞書順、文字列のセットは値の辞書順でまわる（ほかの型のセットは順番の保証なし）。

出典：Conditional Expressions／for Expressions（Element Ordering）

スプラット式と文字列テンプレート

```
var.list[*].id
# = [for o in var.list : o.id]
aws_instance.web[*].public_ip
# count で作ったリソース全部の IP
```

[*] はリスト・セット・タプルに使える。マップ・オブジェクトには使えないので for 式で書く。

リストでない 1 つの値に [*] を付けると要素 1 つのタプル、null なら空のタプル。

```
"Hello, ${var.name}!"          # 埋めこみ
"Hello, %{ if var.name != "" }${var.name}%{ else }unnamed%{ endif }!"
<<EOT
%{ for ip in aws_instance.example[*].private_ip ~}
server ${ip}
%{ endfor ~}
EOT
```

書き方	意味
$${ ・ %%{	文字どおりの ${ ・ %{
~	そちら側の空白・改行を取りのぞく
<<-EOT	インデントを取りのぞくヒアドキュメント

出典：Splat Expressions／Strings and Templates

dynamic ブロック

```
resource "aws_security_group" "web" {
  name = "web"
  dynamic "ingress" {              # 作るブロックの種類
    for_each = var.ingress_rules   # くり返す値
    content {                      # ブロックの中身
      from_port   = ingress.value.port
      to_port     = ingress.value.port
      protocol    = "tcp"
      cidr_blocks = ingress.value.cidrs
    }
  }
}
```

ラベル
作る入れ子のブロックの名前（ingress など）

for_each
コレクション。要素ごとに 1 ブロック

iterator（省略可）
一時変数の名前。既定はラベルと同じ。.key・.value

作れるのは resource・data・provider・provisioner に属する引数のブロックだけ。lifecycle・provisioner などメタ引数のブロックは作れない。

『Overuse of dynamic blocks can make configuration hard to read and maintain』。隠したい細部があるときだけ使う。

出典：dynamic Blocks

第2章

関数

自作はできない。組みこみを組み合わせる。

関数のしくみ

決まり	内容
呼び方	関数名(引数1, 引数2)。max(5, 12, 9) → 12
自作の関数	『You cannot define your own functions in the Terraform configuration language』。プロバイダーの関数は provider::<ローカル名>::<関数名>(…)
引数の展開	min([55, 2453, 2]...) → 2。... はピリオド 3 つ（三点リーダ … ではない）
sensitive	sensitive な引数を使うと結果も sensitive
file・templatefile	設定の最初の検証で読む。plan・apply の途中で作られるファイルは読めない
timestamp・uuid	計画では unknown。apply のときに決まる
試す場所	terraform console（関数のページの例は console の出力）

```
$ terraform console
> cidrsubnet("10.1.2.0/24", 4, 15)
"10.1.2.240/28"
```

出典：Function Calls／Built-in Functions／Command: console

よく使う関数（1）マップと文字列

関数	例	結果・覚えどころ
lookup	lookup({a="ay", b="bee"}, "c", "what?")	"what?"。キーがなければ既定値（m["c"] はエラー）
merge	merge({a="b", c="d"}, {e="f", c="z"})	{a="b", c="z", e="f"}。同じキーは後ろが勝つ
join	join("-", ["foo", "bar", "baz"])	"foo-bar-baz"。区切りが先
split	split(",", "foo,bar,baz")	["foo", "bar", "baz"]。区切りが先
format	format("There are %d lights", 4)	"There are 4 lights"
length	length("hello")・length(["a","b"])	5・2。文字列・リスト・マップ

> 共通のタグ＋個別のタグ：tags = merge(local.common_tags, { Name = "web" })

出典：lookup・merge・join・split・format・length Function

よく使う関数（2）リストと安全な取り出し

関数	例	結果・覚えどころ
element	element(["a", "b", "c"], 3)	"a"。長さを超えると最初にもどる（list[3] はエラー）
contains	contains(["a", "b", "c"], "d")	false。validation でよく使う
coalesce	coalesce("", "b")	"b"。null でも "" でもない最初の値
flatten	flatten([["a", "b"], [], ["c"]])	["a", "b", "c"]。for_each 用に平らに
zipmap	zipmap(["a", "b"], [1, 2])	{a = 1, b = 2}。キーのリスト＋値のリスト
cidrsubnet	cidrsubnet("10.1.2.0/24", 4, 15)	"10.1.2.240/28"。/24 に 4 ビット足して 16 番目
try	try(local.raw.groups, [])	エラーにならない最初の結果
can	can(regex("^[a-z]+$", var.name))	true / false。既定値は返さない

出典：element・contains・coalesce・flatten・zipmap・cidrsubnet・try・can Function

本番での問われ方

ねらい	見るところ
条件式・for 式	型をそろえる、[ ] はタプル・{ } はオブジェクト、if で絞りこみ、... でグループ化
スプラット・テンプレート	[*] と for の書き換え、${ } と %{ }、$${ でエスケープ
dynamic	ラベル・for_each・content・iterator、メタ引数は作れない、使いすぎない
関数のしくみ	自作できない、... で展開、console で試す、file は最初に読む
関数の結果	lookup の既定値、merge は後ろが勝つ、element はくり返す、coalesce・try・can のちがい

出典：Exam Content List（004）4e／本資料の問題演習（全 30 問）